

REGISTRO DE VERSIONAMENTO - SEMANA 17

4. Atividade de Code Review

Comentário de revisão:

Sugestão: verificar o caso em que a lista notas esteja vazia, pois `len(notas)` será igual a zero e isso pode causar um erro na divisão. Também seria interessante utilizar um nome de variável mais claro, como `soma`, para facilitar a compreensão do código.

5. Atividade de resolução de conflito

1. O que o Desenvolvedor A adicionou?

O Desenvolvedor A criou a variável `total`, adicionou uma mensagem exibindo o total calculado e depois retornou o valor.

2. O que o Desenvolvedor B adicionou?

O Desenvolvedor B adicionou uma validação para verificar se o valor é negativo. Caso seja, a função retorna 0.

3. Escolher apenas uma versão faria perder alguma funcionalidade?

Sim. Se fosse escolhida apenas a versão A, perderíamos a validação de valores negativos. Se fosse escolhida apenas a versão B, perderíamos a exibição do total calculado.

4. Versão final preservando as duas melhorias:

```
def calcular_total(valor, taxa):  
    if valor < 0:  
        return 0  
  
    total = valor + (valor * taxa)  
    print("Total calculado:", total)  
  
    return total
```

5. Justificativa técnica:

A versão final combina as duas melhorias sem eliminar nenhuma funcionalidade. A validação do Desenvolvedor B impede que valores negativos sejam processados de forma inadequada, enquanto a alteração do Desenvolvedor A permite armazenar o resultado em uma variável e exibi-lo antes de retorná-lo. Dessa forma, o código mantém as funcionalidades importantes das duas versões.

6. Mensagem de commit sugerida:

Resolve conflito em `calcular_total` e preserva validação e exibição

7. Comentário de Pull Request:

O conflito foi resolvido combinando as duas alterações. A validação para valores negativos foi mantida para evitar resultados inadequados, e a variável total com a exibição do resultado também foi preservada. Assim, a versão final reúne as duas melhorias sem perder funcionalidades.

8. Por que um histórico claro facilita o trabalho da equipe?

Um histórico claro facilita entender quais alterações foram feitas, por quem e por qual motivo. Isso ajuda a identificar problemas, resolver conflitos e acompanhar a evolução do projeto. Commits bem descritos também facilitam a manutenção do código e a colaboração entre os integrantes da equipe.

8. Revisão rápida

1. O que é um fork?

É uma cópia de um repositório que permite trabalhar no projeto sem alterar diretamente o original.

2. Qual é a função do push?

Enviar os commits do computador para o repositório remoto.

3. Para que serve uma Pull Request?

Serve para solicitar que alterações sejam revisadas antes de serem integradas ao projeto principal.

4. Cite dois pontos analisados em um Code Review.

A qualidade do código e a existência de possíveis erros. Também podem ser analisados os nomes das variáveis, organização e facilidade de compreensão.

5. Por que um conflito de merge não deve ser tratado como uma “falha” da equipe?

Porque conflitos são comuns quando várias pessoas trabalham no mesmo projeto. Eles apenas indicam que alterações diferentes precisam ser analisadas e combinadas de maneira adequada.